

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO CUỐI KÌ
PHÁT TRIỂN ỨNG DỤNG CHO CÁC
THIẾT BỊ DI ĐỘNG

Giảng viên hướng dẫn: Nguyễn Hoàng Anh
Sinh viên: Vũ Sỹ Ngọc Hiếu
Mã sinh viên: B22DCCN322
Nhóm lớp học: 07
Nhóm bài tập lớn: 04

Hà Nội – 2026

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	6
CHƯƠNG 1. MỞ ĐẦU.....	7
1.1. Giới thiệu ứng dụng.....	7
1.2. Lý do thực hiện.....	7
1.3. Concept và mục tiêu	7
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ TỔNG QUAN	9
2.1. Kiến trúc tổng quan hệ thống	9
2.2. Phân tích yêu cầu chức năng tổng quan	10
2.3. Biểu đồ Use Case tổng quan.....	10
2.4. Yêu cầu phi chức năng	12
CHƯƠNG 3. THIẾT KẾ CHI TIẾT MODULE CÁ NHÂN	13
3.1. Biểu đồ Use Case chi tiết – Module cá nhân.....	13
3.2. Mô tả chi tiết các Use Case chính.....	14
3.2.1. UC-P01 đến UC-P02 – Thanh toán VNPAY	14
3.2.2. Module Chat Realtime.....	15
3.2.3. Module Thông báo Realtime	17
3.3. Biểu đồ lớp (Class Diagram) – Module cá nhân	18
3.4. Sơ đồ lớp thực thể (ER Diagram).....	21
3.5. Biểu đồ tuần tự (Sequence Diagram)	21
3.5.1. Biểu đồ tuần tự: Thanh toán VNPAY	21
3.5.2. Biểu đồ tuần tự: Hoàn tiền VNPAY tự động.....	23
3.5.3. Biểu đồ tuần tự: Chat Realtime qua WebSocket STOMP.....	24
3.5.4. Biểu đồ tuần tự: Thông báo realtime	25
3.7. Thiết kế giao diện và luồng màn hình	26
CHƯƠNG 4. KẾT QUẢ THỰC HIỆN.....	27
4.1. Mô hình triển khai ứng dụng	27
4.2. Các bước cài đặt và triển khai	27
4.2.1. Backend (Spring Boot)	27
4.2.2. Mobile App (React Native / Expo).....	28
4.3. Kết quả thực hiện – Các chức năng Module cá nhân	28
4.3.1. Chức năng Thanh toán VNPAY	28
4.3.2. Chức năng Chat Realtime.....	29
4.3.3. Chức năng Thông báo Realtime	31
4.4. Kết quả thử nghiệm	33
4.5. Dữ liệu thử nghiệm trong hệ thống	34

CHƯƠNG 5. KẾT LUẬN	35
5.1. Kết quả đạt được.....	35
5.2. Hạn chế và điểm cần cải thiện	35
5.3. Hướng phát triển.....	35
CHƯƠNG 6. PHỤ LỤC: BÁO CÁO KỸ THUẬT	37
6.1. Các lớp Entity và Bảng trong CSDL	37
6.1.1. Entity Payment – Bảng payment (MySQL)	37
6.1.2. Entity Booking – Bảng booking (MySQL)	38
6.1.3. ChatThreadDocument – MongoDB collection: chat_threads	38
6.1.4. ChatMessageDocument – MongoDB collection: chat_messages	39
6.2. Các Controller – Lớp xử lý HTTP Request.....	40
6.2.1. CustomerBookingController (phần Thanh toán VNPAY)	40
6.2.2. ChatController	41
6.3. Các Service – Lớp xử lý nghiệp vụ	41
6.3.1. VnPayService	41
6.3.2. CustomerBookingService (phần thanh toán).....	42
6.3.3. ChatService.....	44
6.3.4. ChatRealtimePublisher	45
6.3.5. NotificationRealtimePublisher	45
6.4. Các Repository – Lớp truy cập CSDL.....	46
6.4.1. ChatThreadRepository (MongoDB)	46
6.4.2. ChatMessageRepository (MongoDB)	47
6.4.3. BookingRepository (MySQL – liên quan thanh toán).....	47
6.5. Các DTO (Data Transfer Object)	47
6.6. Các API Gọi Ngoài (External API)	48
6.6.1. VNPAY Payment Gateway	48
6.6.2. WebSocket STOMP – Chat và Thông báo Realtime	49

DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc tổng quan hệ thống Ride Up	9
Hình 2. Biểu đồ Use Case tổng quan hệ thống.....	11
Hình 3. Biểu đồ Use Case chi tiết – Module cá nhân.....	13
Hình 4. Biểu đồ lớp – Module cá nhân.....	19
Hình 5. Luồng màn hình thanh toán VNPay trên ứng dụng.....	29
Hình 6. Luồng màn hình chat realtime phía khách hàng.....	30
Hình 7. Luồng màn hình chat realtime phía tài xế	31
Hình 8. Màn hình thông báo realtime phía tài xế	32
Hình 9. Màn hình thông báo realtime phía khách hàng.....	33

DANH MỤC BẢNG BIỂU

Bảng 1. Danh sách thành viên và phân công nhiệm vụ.....	5
Bảng 2. Bảng danh mục từ viết tắt	6
Bảng 3. Các thành phần kiến trúc hệ thống	10
Bảng 4. Yêu cầu chức năng tổng quan theo actor	10
Bảng 5. Danh sách Use Case module cá nhân.....	13
Bảng 6. Mô tả Use Case UC-P01, UC-P02: Tạo và xác nhận thanh toán VNPay	15
Bảng 7. Mô tả Use Case UC-P03: hoàn tiền tự động khi hủy	15
Bảng 8. Mô tả Use Case UC-C01, UC-C02: Tạo chat và gửi nhắn tin realtime	16
Bảng 9. Mô tả Use Case UC-C04, UC-C05: Lịch sử chat và đóng đoạn chat	17
Bảng 10. Mô tả Use Case UC-N01, UC-N02: Nhận và xem thông báo Realtime	18
Bảng 11. Bảng diễn giải biểu đồ lớp	21
Bảng 12. Thiết kế giao diện – Module cá nhân	26
Bảng 13. Mô hình triển khai ứng dụng.....	27
Bảng 14. Kết quả kiểm thử.....	34
Bảng 15. Dữ liệu thử nghiệm trong hệ thống	34

DANH SÁCH THÀNH VIÊN VÀ ĐÓNG GÓP

STT	Họ và tên	MSSV	Đóng góp cụ thể
1	Phạm Quang Huy	B22DCCN394	Tài xế: Đăng ký/cập nhật hồ sơ tài xế, tạo chuyến đi, quản lý trạng thái chuyến, xem danh sách & chi tiết chuyến, xem danh sách booking, xác nhận thanh toán tiền mặt
2	Nguyễn Xuân Hưng	B22DCCN418	Tìm kiếm chuyến , danh sách booking của user, chatbot hỗ trợ, đặt xe
3	Vũ Sỹ Ngọc Hiếu	B22DCCN322	Thanh toán trực tuyến bằng cổng VNPAY, Hoàn tiền tự động khi thực hiện hủy chuyến, Nhắn tin trực tuyến, Thông báo realtime cho các sự kiện
4	Khúc Văn Tuấn	B22DCCN754	Đăng kí , đăng nhập, Cập nhật thông tin người dùng, Đánh giá chuyến
5	Nguyễn Vũ Tuấn Khôi	B22DCCN466	Báo cáo thống kê, tất cả các chức năng phía role admin (duyet hồ sơ, quản lý user, quản lý thông tin địa lý hành chính)

Bảng 1. Danh sách thành viên và phân công nhiệm vụ

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
API	Application Programming Interface – Giao diện lập trình ứng dụng
REST	REpresentational State Transfer – Phong cách thiết kế kiến trúc web service
JWT	JSON Web Token – Chuẩn xác thực và truyền thông tin qua token mã hóa
WebSocket	Giao thức kết nối hai chiều liên tục giữa client và server
STOMP	Simple Text Oriented Messaging Protocol – Giao thức nhắn tin trên WebSocket
SSE	Server-Sent Events – Kỹ thuật server đẩy sự kiện realtime tới client (một chiều)
VNPay	Vietnam Payment – Cổng thanh toán trực tuyến của Việt Nam
HMAC	Hash-based Message Authentication Code – Mã xác thực bằng hàm băm
IPN	Instant Payment Notification – Thông báo thanh toán tức thời từ cổng thanh toán
DB	Database – Cơ sở dữ liệu
DTO	Data Transfer Object – Đối tượng truyền dữ liệu giữa các tầng
ORM	Object-Relational Mapping – Ánh xạ đối tượng và bảng quan hệ
UI	User Interface – Giao diện người dùng
UX	User Experience – Trải nghiệm người dùng
MAD	Mobile Application Development – Lập trình ứng dụng di động

Bảng 2. Bảng danh mục từ viết tắt

CHƯƠNG 1. MỞ ĐẦU

1.1. Giới thiệu ứng dụng

Ride Up là ứng dụng di động phục vụ nhu cầu đặt xe liên tỉnh theo mô hình ghép chỗ (ride-sharing). Ứng dụng cho phép tài xế có phương tiện cá nhân đăng ký và tạo các chuyến đi liên tỉnh, đồng thời cho phép hành khách tìm kiếm và đặt chỗ trực tiếp trên ứng dụng với mức giá cố định và linh hoạt về lịch trình.

Ứng dụng được phát triển trên nền tảng React Native (Expo) cho phần giao diện di động và Spring Boot 3.x cho phần backend, sử dụng cơ sở dữ liệu , MongoDB cho dữ liệu chat, Redis cho cache và VNPay cho thanh toán trực tuyến.

Hệ thống hỗ trợ ba nhóm người dùng: Tài xế (Driver) – tạo và quản lý chuyến đi; Hành khách (Customer) – tìm kiếm và đặt chỗ; Quản trị viên (Admin) – duyệt hồ sơ tài xế và quản lý hệ thống.

1.2. Lý do thực hiện

Nhu cầu di chuyển liên tỉnh tại Việt Nam rất lớn. Các phương tiện truyền thống như xe khách, xe giường nằm và tàu hỏa mặc dù phổ biến nhưng còn nhiều hạn chế: giá vé không linh hoạt, điểm đón trả cố định, và lịch trình không thể điều chỉnh. Trong khi đó, hàng nghìn tài xế có nhu cầu di chuyển liên tỉnh mỗi ngày nhưng chưa có nền tảng để kết nối và chia sẻ chỗ trống.

Ứng dụng Ride Up được thực hiện nhằm giải quyết những vấn đề đó, tạo ra một nền tảng kết nối tài xế và hành khách theo hướng chia sẻ chỗ đi liên tỉnh, giúp giảm chi phí cho cả hai bên và tận dụng nguồn lực phương tiện hiện có.

1.3. Concept và mục tiêu

Ứng dụng được thiết kế theo mô hình marketplace hai chiều: tài xế tự đăng chuyến đi kèm giá cố định trên từng ghế, hành khách tìm kiếm và đặt chỗ trên các chuyến đó. Mục tiêu cụ thể:

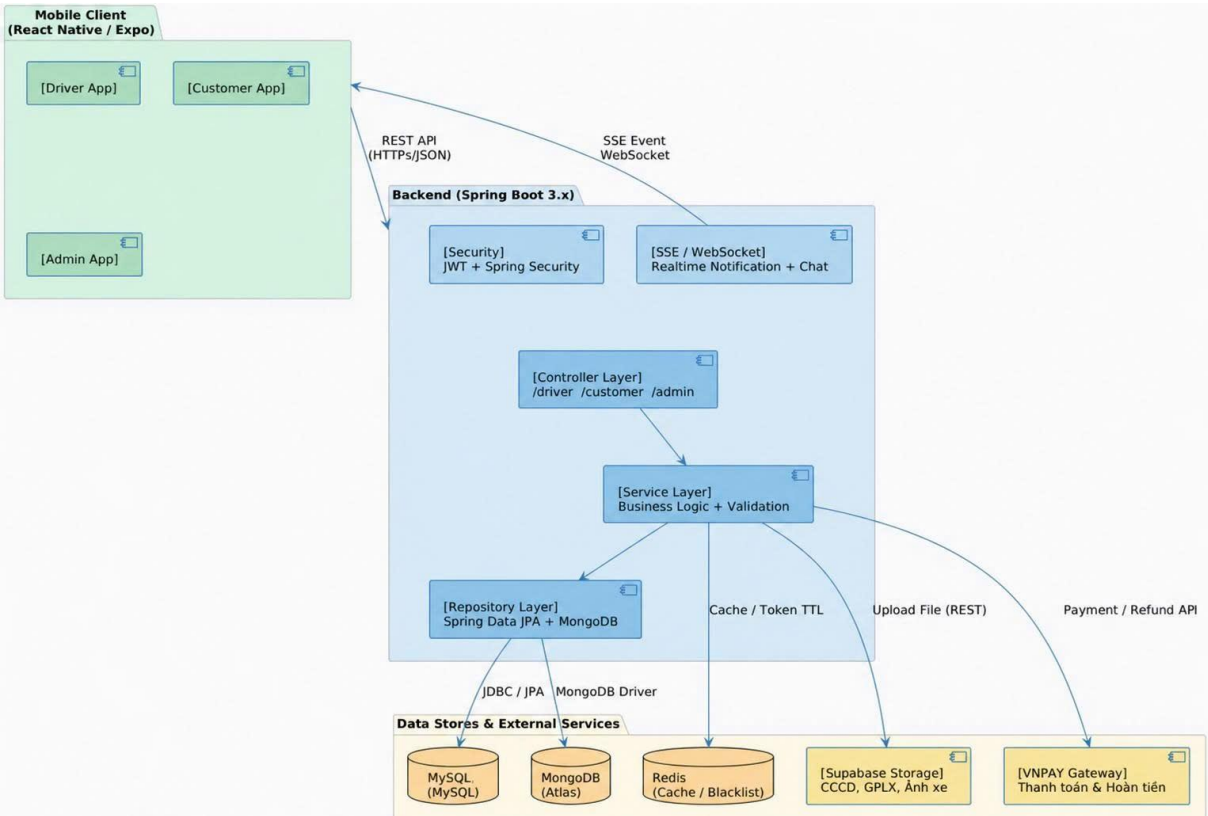
- Xây dựng nền tảng mobile dễ sử dụng, chạy được trên cả Android và iOS

- Xác minh tài xế qua quy trình duyệt hồ sơ (CCCD, GPLX, thông tin xe) bởi Admin
- Hỗ trợ thanh toán online qua VNPay và tiền mặt với hoàn tiền tự động khi hủy chuyến
- Cung cấp thông báo và nhắn tin realtime giữa tài xế và hành khách
- Đảm bảo bảo mật qua xác thực JWT và phân quyền theo role

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ TỔNG QUAN

2.1. Kiến trúc tổng quan hệ thống

Hệ thống Ride Up được xây dựng theo mô hình kiến trúc Client – Server nhiều tầng, trong đó phần giao diện di động và phần backend hoàn toàn tách biệt và giao tiếp qua REST API. Hình 1 mô tả tổng quan kiến trúc hệ thống.



Hình 1. Kiến trúc tổng quan hệ thống Ride Up

Mô tả các thành phần:

Thành phần	Công nghệ	Mô tả vai trò
Mobile Client	React Native / Expo	Giao diện người dùng cho cả 3 role (Driver, Customer, Admin). Gọi REST API, kết nối SSE realtime và WebSocket chat.
Backend API	Spring Boot 3.x	Xử lý toàn bộ logic nghiệp vụ, phân quyền JWT theo role, expose các REST endpoint, xử lý thanh toán VNPay, publish sự kiện SSE.
MySQL	Supabase (cloud)	Lưu trữ dữ liệu quan hệ: User, DriverProfile, Vehicle, Trip,

		TripPickupPoint, TripDropoffPoint, Booking, Payment, BookingReview, Ward, Province.
MongoDB	MongoDB Atlas	Lưu trữ dữ liệu chat: ChatThreadDocument (luồng hội thoại) và ChatMessageDocument (tin nhắn).
Redis	Upstash / Railway	Cache API response để giảm tải DB, lưu trữ danh sách refresh token bị vô hiệu hóa, quản lý session realtime.
Supabase Storage	Supabase (cloud)	Lưu trữ file tải lên: ảnh CCCD (mặt trước/sau), ảnh GPLX, avatar, ảnh xe, đăng ký xe, bảo hiểm.
VNPay Gateway	VNPay Sandbox/Live	Xử lý thanh toán trực tuyến và hoàn tiền (refund) tự động khi tài xế hoặc khách hủy chuyến.
SSE / WebSocket	Spring (built-in)	SSE đẩy thông báo realtime đến client; WebSocket cho tính năng chat realtime hai chiều.

Bảng 3. Các thành phần kiến trúc hệ thống

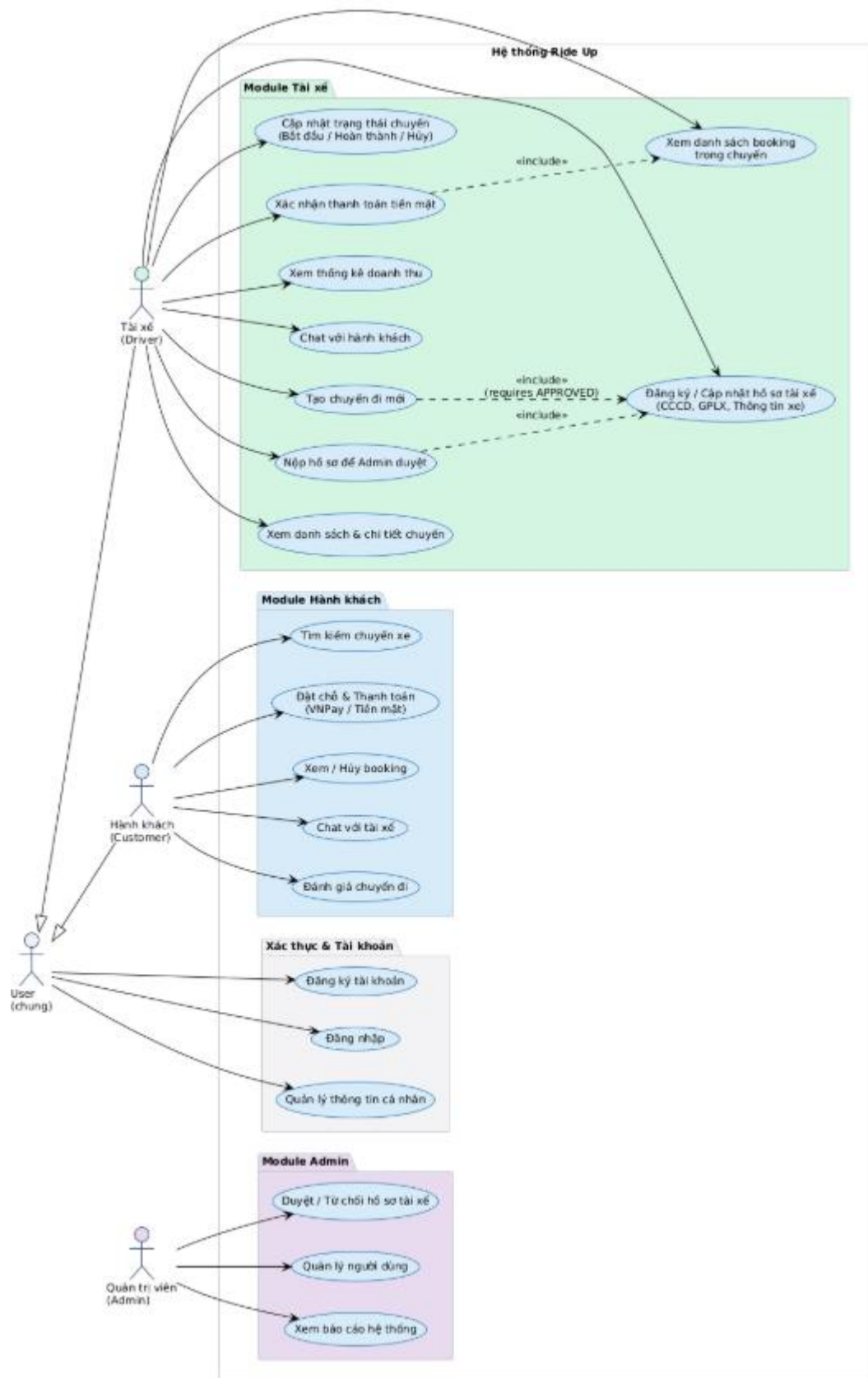
2.2. Phân tích yêu cầu chức năng tổng quan

Hệ thống Ride Up có ba actor chính với các nhóm chức năng tương ứng:

Actor	Nhóm chức năng
Tài xế (Driver)	Đăng ký/cập nhật hồ sơ; tạo chuyến đi; quản lý chuyến (bắt đầu, hoàn thành, hủy); xem danh sách & chi tiết chuyến; xem danh sách booking; xác nhận thanh toán tiền mặt; thống kê doanh thu; chat realtime.
Hành khách (Customer)	Tìm kiếm chuyến xe; xem chi tiết chuyến; đặt chỗ và thanh toán; xem/hủy booking; chat với tài xế; đánh giá chuyến.
Quản trị viên (Admin)	Xem danh sách hồ sơ tài xế chờ duyệt; duyệt/từ chối hồ sơ; quản lý người dùng; xem báo cáo thống kê hệ thống.

Bảng 4. Yêu cầu chức năng tổng quan theo actor

2.3. Biểu đồ Use Case tổng quan



Hình 2. Biểu đồ Use Case tổng quan hệ thống

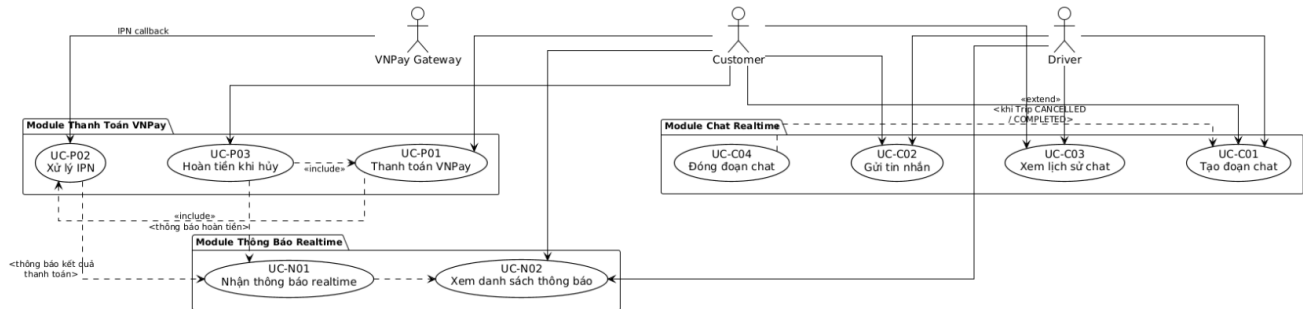
Biểu đồ Use Case tổng quan thể hiện mối quan hệ giữa ba actor chính (Driver, Customer, Admin) và các nhóm chức năng tương ứng. Actor Driver và Customer kế thừa chức năng từ actor User (đăng nhập, đăng ký, quản lý tài khoản). Admin có quyền truy cập vào các chức năng quản lý riêng biệt không chồng chéo với Driver hay Customer.

2.4. Yêu cầu phi chức năng

- Hiệu năng: API phải trả về response dưới 500ms với hỗ trợ cache Redis; hệ thống đảm bảo tối thiểu 100 người dùng đồng thời
- Bảo mật: Xác thực bằng JWT (access token + refresh token), phân quyền role-based (DRIVER/CUSTOMER/ADMIN); mật khẩu mã hóa BCrypt
- Tính khả dụng: Ứng dụng mobile chạy được trên cả Android (API 26+) và iOS (12+); backend triển khai trên cloud với uptime cao
- Khả năng mở rộng: Kiến trúc layered (Controller – Service – Repository) dễ bảo trì và mở rộng; MongoDB Atlas và MySQL Supabase có khả năng scale tốt
- Trải nghiệm người dùng: Giao diện phản hồi nhanh với skeleton loading; thông báo realtime tức thì qua SSE

CHƯƠNG 3. THIẾT KẾ CHI TIẾT MODULE CÁ NHÂN

3.1. Biểu đồ Use Case chi tiết – Module cá nhân



Hình 3. Biểu đồ Use Case chi tiết – Module cá nhân

Mã UC	Tên Use Case	Mô tả ngắn
UC-P01	Thanh toán VNPAY	Hành khách chọn thanh toán VNPAY Hệ thống tạo đường dẫn đã ký (HMAC-SHA512) và điều hướng sang cổng VNPAY
UC-P02	Xử lý xác nhận thanh toán (IPN)	VNPAY gọi callback IPN về hệ thống Hệ thống xác minh chữ ký và cập nhật trạng thái thanh toán và booking
UC-P03	Hoàn tiền khi hủy giao dịch	Khi chuyến hoặc booking bị hủy, hệ thống tự động gửi lệnh hoàn tiền VNPAY và cập nhật trạng thái thanh toán thành đã hoàn tiền
UC-C01	Tạo đoạn chat khi booking xác nhận	Đoạn chat giữa tài xế và hành khách được tạo tự động ngay khi chuyến được đặt thành công
UC-C02	Gửi tin nhắn	Người dùng gửi tin nhắn, hệ thống lưu vào MongoDB và push realtime đến đối phương
UC-C03	Xem lịch sử chat	Tải tin nhắn cũ theo thứ tự từ mới đến cũ
UC-C04	Đóng đoạn chat khi chuyến kết thúc	Khi chuyến hoàn thành hoặc bị hủy, đoạn chat của chuyến sẽ kết thúc, không nhắn thêm được
UC-N01	Nhận thông báo realtime	Nhận thông báo từ các sự kiện như booking, chuyến, thanh toán thành công,...
UC-N02	Xem danh sách thông báo	Người dùng mở màn hình Thông báo, xem lại toàn bộ sự kiện đã nhận và điều hướng đến màn hình liên quan

Bảng 5. Danh sách Use Case module cá nhân

3.2. Mô tả chi tiết các Use Case chính

3.2.1. UC-P01 đến UC-P02 – Thanh toán VNPay

Thuộc tính	Nội dung
Use Case	UC-P01: Thanh toán VNPay UC-P02: Xử lý xác nhận thanh toán (IPN)
Actor	UC-P01: Hành khách UC-P02: VNPay Gateway
Điều kiện tiên quyết	Hành khách đã đặt chỗ thành công và chọn hình thức VNPay; tài khoản merchant VNPay đã được cấu hình (tmnCode, hashSecret, returnUrl)
UC-P01: Tạo URL thanh toán	1. Hành khách nhấn "Thanh toán VNPay" trên màn hình xác nhận đặt chỗ. 2. Hệ thống tạo booking, ký xác thực và trả đường dẫn VNPay đã ký về ứng dụng; 3. Ứng dụng mở màn hình WebView dẫn đến trang thanh toán VNPay. 4. Hành khách chọn ngân hàng và hoàn tất xác thực OTP trực tiếp trên giao diện VNPay.
UC-P02 Xử lý xác nhận IPN	1. Hành khách thực hiện thanh toán xong, VNPay gọi về hệ thống để thông báo kết quả. 2. Hệ thống kiểm tra chữ ký của yêu cầu và tra cứu bản ghi thanh toán tương ứng với mã 3. Nếu thanh toán thành công: trạng thái thanh toán cập nhật thành Đã thanh toán, booking chuyển sang Đã xác nhận, ghi nhận mã giao dịch VNPay và thời điểm thanh toán. 4. Nếu thanh toán thất bại: trạng thái thanh toán cập nhật thành Thất bại, booking giữ nguyên. 5. Hệ thống gửi thông báo realtime đến hành khách về kết quả giao dịch. 6. Hệ thống phản hồi về VNPay để xác nhận đã xử lý thành công.
Hậu điều kiện	Trạng thái thanh toán và booking được cập nhật chính xác; hành khách nhận thông báo kết quả

API liên quan	POST /bookings/payment/initiate – tạo URL POST /vnpay/ipn – nhận callback từ VNPay GET /vnpay/callback – xử lý return URL sau thanh toán
----------------------	--

Bảng 6. Mô tả Use Case UC-P01, UC-P02: Tạo và xác nhận thanh toán VNPay

Thuộc tính	Nội dung
Mã Use Case	UC-P03
Tên	Hoàn tiền khi hủy giao dịch
Actor	Tài xế hoặc Hành khách
Điều kiện tiên quyết	Booking/chuyến bị hủy và trạng thái thanh toán là đã thanh toán, phương thức thanh toán VNPay
UC-P03: Hoàn tiền khi hủy	1. Tài xế bấm hủy chuyến hoặc hành khách bấm hủy booking đã thanh toán bằng VNPay. 2. Hệ thống tìm tất cả các booking trong chuyến (hoặc booking đó) có trạng thái thanh toán là đã thanh toán, phương thức thanh toán VNPay. 3. Với mỗi booking cần hoàn tiền, hệ thống gửi lệnh hoàn tiền toàn phần đến VNPay kèm mã giao dịch gốc và chữ ký xác thực. 4. Nếu VNPay xác nhận hoàn tiền thành công: trạng thái thanh toán chuyển sang Đã hoàn tiền, ghi nhận thời điểm hoàn. 5. Hành khách nhận thông báo realtime kết quả: hoàn tiền thành công.
Hậu điều kiện	Tiền hoàn về tài khoản hành khách khi hủy hợp lệ.
API liên quan	PUT /driver/trips/{id}/cancel PUT /customer/bookings/{id}/cancel

Bảng 7. Mô tả Use Case UC-P03: hoàn tiền tự động khi hủy

3.2.2. Module Chat Realtime

Thuộc tính	Nội dung
Mã Use Case	UC-C01 / UC-C02
Tên	Tạo đoạn chat / Gửi tin nhắn
Actor	Tài xế (Driver), Hành khách (Customer)

Điều kiện tiên quyết	Booking tồn tại và đã được xác nhận, cả hai phía đã đăng nhập và ứng dụng đang mở.
UC-C01 Tạo đoạn chat	1. Tài xế xác nhận booking 2. Hệ thống tạo một đoạn chat mới gắn với booking, lưu ID tài xế, ID hành khách và đánh dấu trạng thái Mở. 3. Cả hai phía có thể mở màn hình Chat bất cứ lúc nào để bắt đầu hoặc tiếp tục nhắn tin.
UC-C02: Gửi tin nhắn	1. Tài xế hoặc hành khách nhập nội dung tin nhắn và nhấn Gửi. 2. Hệ thống kiểm tra đoạn chat còn ở trạng thái Mở (chưa bị đóng do chuyển kết thúc), nếu còn thì tin nhắn được lưu vào CSDL và gửi đi. 3. Hệ thống cập nhật thông tin đoạn chat: nội dung xem trước, thời điểm tin nhắn cuối và tăng số tin chưa đọc của phía nhận lên một. 4. Tin nhắn được gửi ngay đến người đang theo dõi đoạn chat.
Hậu điều kiện	Tin nhắn được lưu bền vững trong MongoDB; cả hai phía thấy nội dung đồng nhất theo thời gian thực
API liên quan	POST /chat/threads – tạo thread GET /chat/threads – lấy danh sách WebSocket /ws – STOMP endpoint /app/chat – gửi tin nhắn, /topic/chat.thread.{threadId} – nhận tin nhắn

Bảng 8. Mô tả Use Case UC-C01, UC-C02: Tạo chat và gửi nhắn tin realtime

Thuộc tính	Nội dung
Mã Use Case	UC-C03 / UC-C04
Tên	Xem lịch sử chat / Đóng đoạn chat khi chuyển kết thúc
Actor	Tài xế (Driver), Hành khách (Customer)
Điều kiện tiên quyết	UC-C04: Đoạn chat đã có ít nhất một tin nhắn UC-C05: Chuyến đã hoàn thành hoặc bị hủy
UC-C04 Xem lịch sử chat	1. Người dùng mở màn hình Chat 2. Hệ thống tự động tải 20 tin nhắn gần nhất. 3. Người dùng cuộn lên đến đầu danh sách 4. Hệ thống tải thêm 20 tin nhắn cũ hơn. Toàn bộ lịch sử được hiển thị theo thứ tự thời gian, tin mới

	nhất ở dưới cùng. Số tin chưa đọc của đoạn chat được đặt lại về không.
UC-C05 Đóng đoạn chat	1. Tài xế hoàn thành hoặc hủy chuyến 2. Hệ thống tự động đóng tất cả đoạn chat liên quan đến chuyến đó. Ô nhập tin nhắn trên ứng dụng bị vô hiệu hóa. Lịch sử trò chuyện vẫn được giữ nguyên; cả hai phía có thể xem lại bất cứ lúc nào
Hậu điều kiện	Lịch sử chat lưu đầy đủ trong MongoDB; đoạn chat bị đóng khi chuyến kết thúc; không thể gửi thêm tin
API liên quan	GET /chat/threads/{threadId}/messages – tải lịch sử PUT /chat/threads/{threadId}/read – đánh dấu đã đọc

Bảng 9. Mô tả Use Case UC-C04, UC-C05: Lịch sử chat và đóng đoạn chat

3.2.3. Module Thông báo Realtime

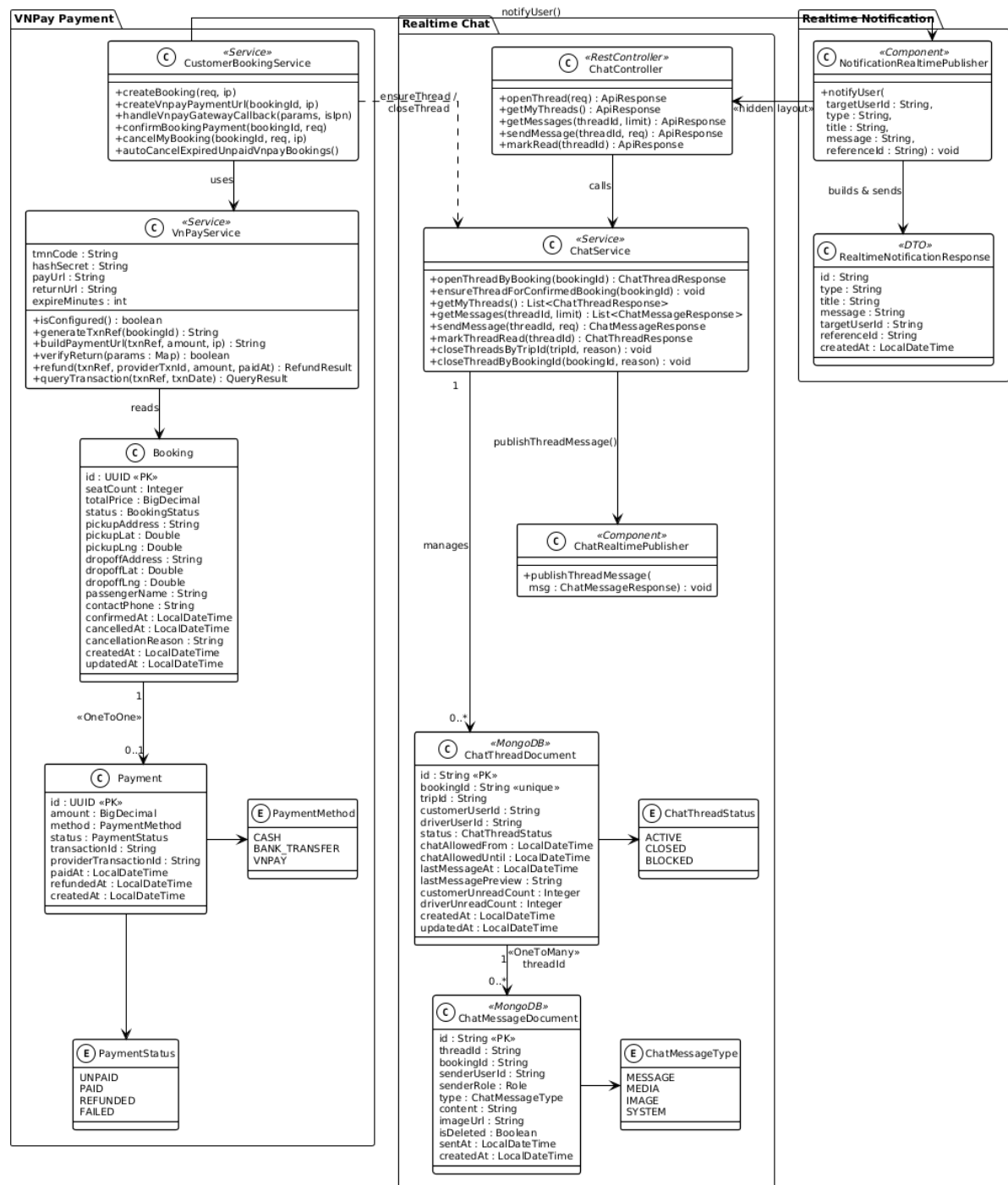
Thuộc tính	Nội dung
Use Case	UC-N01: Nhận thông báo realtime UC-N02: Xem danh sách thông báo
Actor	Tài xế, Hành khách
Điều kiện tiên quyết	Người dùng đã đăng nhập và ứng dụng đang mở; kết nối WebSocket còn hoạt động
UC-N01 Nhận thông báo realtime	1. Khi ứng dụng khởi động, hệ thống thiết lập kết nối WebSocket và đăng ký nhận thông báo cho tài khoản đang đăng nhập. 2. khi có sự kiện liên quan, hệ thống đóng gói thông báo gồm: mã định danh, loại sự kiện, tiêu đề, nội dung, mã tham chiếu (bookingId/tripId) và thời điểm phát sinh. 3. Thông báo được đẩy ngay đến client đang kết nối, hiển thị dưới dạng banner nổi trên màn hình hiện tại; biểu tượng chuông cập nhật số chưa đọc.
Các sự kiện kích hoạt	Đối với Hành khách: • Tài xế xác nhận booking → "Booking của bạn đã được xác nhận" • Thanh toán VNPay thành công → "Thanh toán thành công" • Tài xế xác nhận nhận tiền mặt → "Tài xế đã xác nhận thanh toán" • Chuyến đi bị hủy → "Chuyến đã bị hủy"

	- Tài xế hủy booking → "Booking của bạn đã bị hủy" Đối với Tài xế: - Hành khách đặt chỗ mới → "Có booking mới trên chuyến của bạn" - Hành khách hủy booking → "Hành khách đã hủy đặt chỗ" - Hồ sơ tài xế được duyệt / từ chối → "Hồ sơ của bạn đã được xử lý"
UC-N02 Xem danh sách thông báo	- Người dùng nhấn vào biểu tượng chuông để mở màn hình Thông báo. - Hệ thống hiển thị danh sách toàn bộ thông báo hiển thị theo thứ tự thời gian; mỗi dòng gồm loại sự kiện, tiêu đề, nội dung tóm tắt và thời gian nhận. Tất cả thông báo chưa đọc được đánh dấu đã đọc sau khi mở màn hình; số hiển thị trên chuông về không.
Hậu điều kiện	Người dùng nắm được mọi diễn biến liên quan đến booking, chuyến đi và thanh toán mà không cần chủ động kiểm tra
API liên quan	WebSocket /ws – STOMP endpoint /topic/notifications.user.{userId} – subscribe nhận thông báo GET /notifications – lấy lịch sử thông báo (nếu có persistence)

Bảng 10. Mô tả Use Case UC-N01, UC-N02: Nhận và xem thông báo Realtime

3.3. Biểu đồ lớp (Class Diagram) – Module cá nhân

Biểu đồ lớp mô tả các entity, document và service class chính trong module cá nhân. Phần thanh toán sử dụng MySQL, phần chat sử dụng MongoDB Document.



Hình 4. Biểu đồ lớp – Module cá nhân

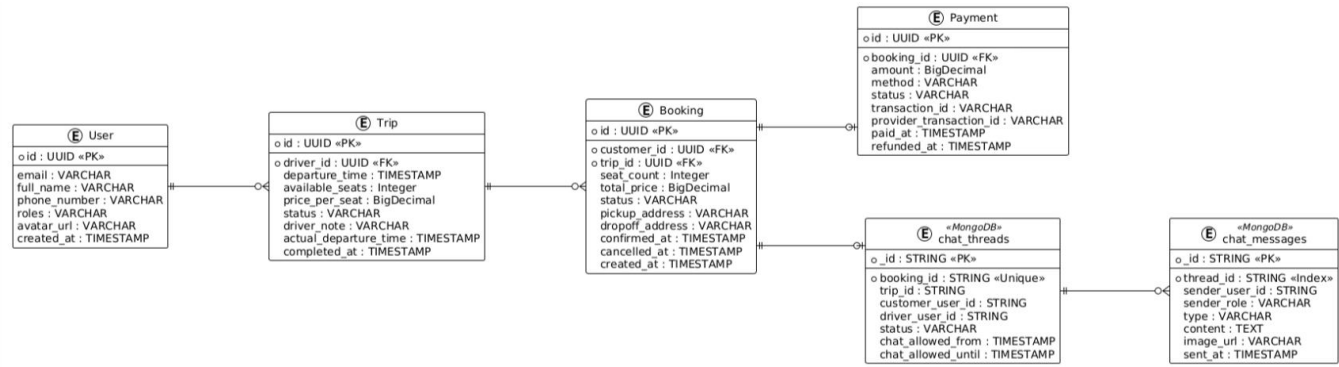
Module	Lớp / Thành phần	Loại	Chức năng & Mô tả ngắn gọn
Thanh toán VNPay	VNPayService	Service	Xử lý tích hợp API VNPAY: tạo chữ ký HMAC, sinh URL thanh toán, verify callback, hoàn tiền (refund).

	CustomerBooking Service	Service	Điều phối toàn bộ luồng đặt xe: gọi VNPay để thanh toán, gọi Notification để thông báo, gọi Chat để mở thread.
	Booking	JPA Entity	Bảng lưu đơn đặt xe. Liên kết 1-1 với thanh toán và kênh chat.
	Payment	JPA Entity	Bảng lưu giao dịch thanh toán (thuộc tính: số tiền, mã giao dịch, thời gian trả). Quan hệ 1-1 với Booking.
	PaymentMethod/ PaymentStatus	Enum	Liệt kê các hình thức (CASH, VNPAY...) và trạng thái (UNPAID, PAID, REFUNDED...) thanh toán.
Thông báo realtime	NotificationRealtime Publisher	Component	Gửi thông báo (event) tức thì qua WebSocket/STOMP tới từng user.
	RealtimeNotification Response	DTO	Payload dữ liệu chứa nội dung thông báo sẽ được push xuống ứng dụng (React Native).
Chat realtime	ChatController	RestController	Các API endpoint để mở kênh chat, gửi tin nhắn, đánh dấu đã đọc.
	ChatService	Service	Xử lý logic nghiệp vụ chat: kiểm tra quyền, lưu MongoDB, gọi ChatRealtimePublisher để đẩy tin nhắn.
	ChatRealtimePublisher	Component	Phát tin nhắn realtime qua WebSocket/STOMP tới kênh của thread /topic/chat/thread/{threadId}.

	ChatThreadDocument	MongoDB Document	Đại diện cho 1 phòng chat giữa Khách - Tài xế (thuộc 1 booking duy nhất).
	ChatMessageDocument	MongoDB Document	Lưu trữ nội dung từng tin nhắn (text/ảnh). Quan hệ N-1 với ChatThreadDocument.
	ChatThreadStatus/ ChatMessageType	Enum	Các trạng thái của kênh chat (ACTIVE, CLOSED) và loại tin nhắn (MESSAGE, MEDIA).

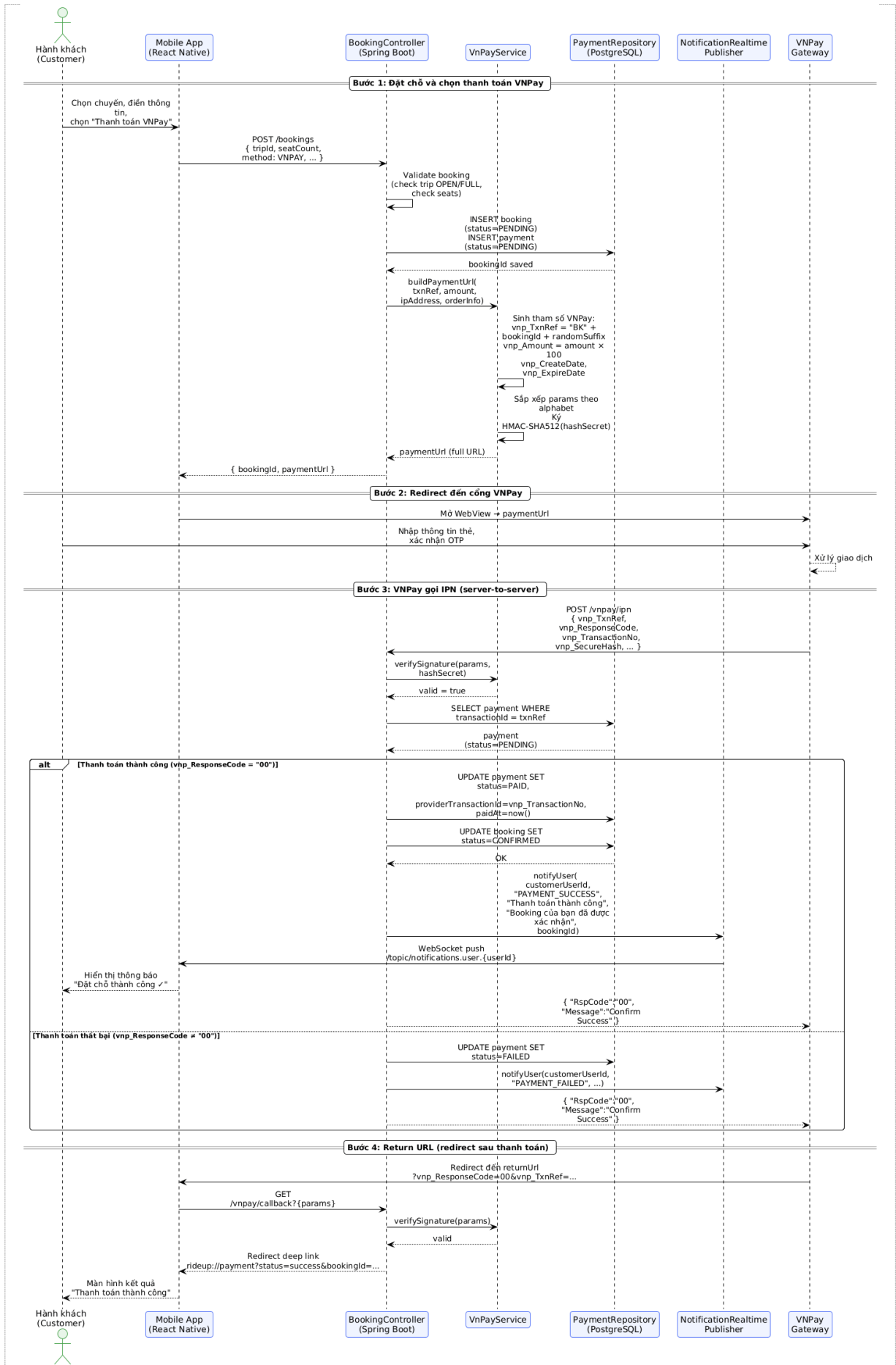
Bảng 11. Bảng diễn giải biểu đồ lớp

3.4. Sơ đồ lớp thực thể (ER Diagram)

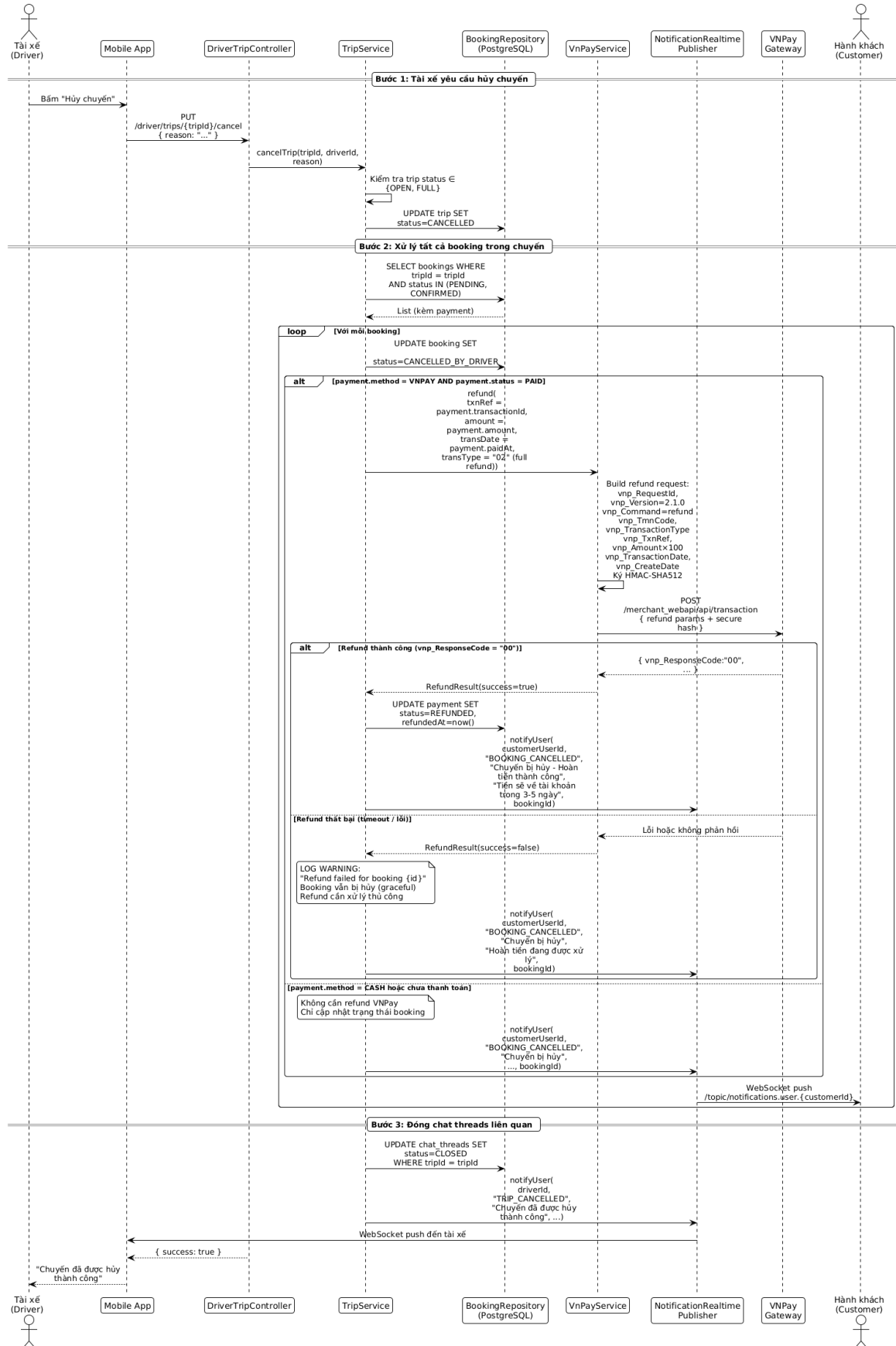


3.5. Biểu đồ tuần tự (Sequence Diagram)

3.5.1. Biểu đồ tuần tự: Thanh toán VNPay



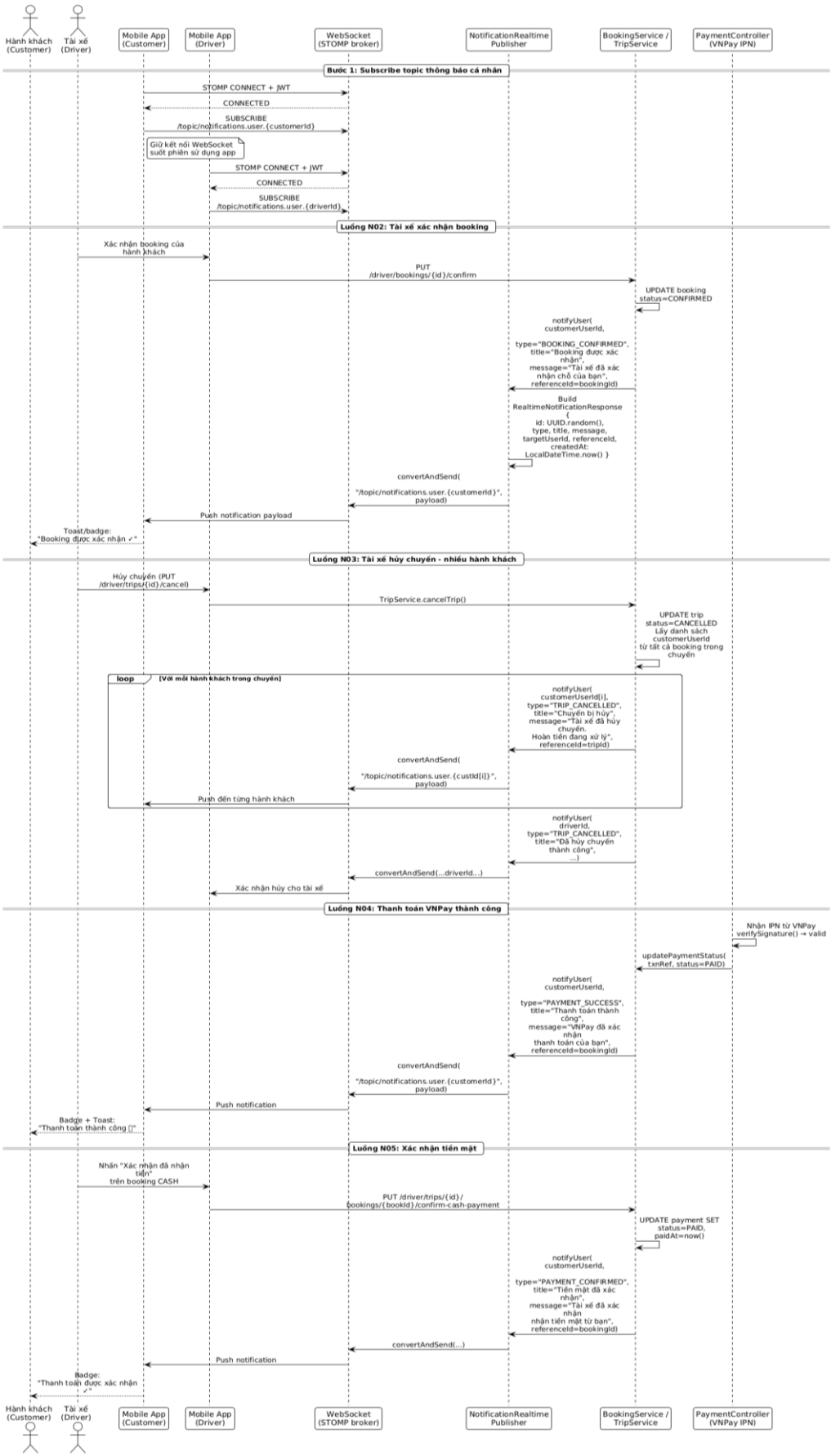
3.5.2. Biểu đồ tuần tự: Hoàn tiền VNPay tự động



3.5.3. Biểu đồ tuần tự: Chat Realtime qua WebSocket STOMP



3.5.4. Biểu đồ tuần tự: Thông báo realtime



3.7. Thiết kế giao diện và luồng màn hình

Giao diện module cá nhân được xây dựng theo theme màu xanh lá nhất quán với toàn bộ ứng dụng. Dưới đây là mô tả các màn hình chính:

Màn hình	Component File	Chức năng và thành phần UI
Danh sách Chat (Driver)	DriverMessagesScreen.js	Danh sách chat threads; số tin chưa đọc; hiển thị tin nhắn cuối; avatar hành khách; kết nối WebSocket tự động; realtime cập nhật tin nhắn
Danh sách Chat (Customer)	CustomerMessagesScreen.js	Tương tự DriverMessagesScreen nhưng hiển thị thông tin tài xế; sắp xếp theo tin nhắn mới nhất tới cũ nhất
Màn hình Chat	ChatScreen.js	Tin nhắn phân biệt người gửi (bên phải, nền xanh) và người nhận (bên trái, nền xám); ô nhập tin nhắn + nút gửi; hiển thị tin nhắn cũ; header hiển thị tên đối phương
Thanh toán VNPAY	PaymentScreen.js / WebView	Chọn phương thức (VNPay/Cash); nút "Thanh toán VNPAY", app điều hướng đến URL VNPAY; xử lý return URL; hiển thị kết quả (thành công/thất bại)
Thông báo	NotificationCenterScreen.js	Danh sách thông báo realtime theo thời gian; hiển thị tổng số thông báo chưa đọc; nhấn vào thông báo thì điều hướng đến màn hình liên quan

Bảng 12. Thiết kế giao diện – Module cá nhân

CHƯƠNG 4. KẾT QUẢ THỰC HIỆN

4.1. Mô hình triển khai ứng dụng

Thành phần	Nền tảng	Ghi chú kỹ thuật
Backend Spring Boot	Railway / Render / VPS	Java 17, Maven; expose REST API & WebSocket trên cổng 8080; kết nối MySQL, MongoDB, Redis qua biến môi trường
MySQL	MySQL Workbench	Lưu Payment và dữ liệu quan hệ khác
MongoDB	MongoDB Atlas (M0 free)	Lưu ChatThreadDocument và ChatMessageDocument; index trên threadId, bookingId, sentAt
Redis	Upstash / Railway Redis	Cache API response; lưu blacklisted refresh token; TTL theo module
VNPay Gateway	VNPay Sandbox / Live	tmnCode + hashSecret trong application.yml; returnUrl + ipnUrl cấu hình tại merchant portal
Mobile App	Expo Go / APK build	React Native 0.74+; WebSocket qua thư viện @stomp/stompjs + sockjs-client; kết nối qua BASE_URL trong config.js

Bảng 13. Mô hình triển khai ứng dụng

4.2. Các bước cài đặt và triển khai

4.2.1. Backend (Spring Boot)

- Yêu cầu: Java 17+, Maven 3.8+
- Clone source code: `git clone https://github.com/hungnx-yb/Ride_Up.git`
- Vào thư mục backend: `cd Ride_Up/RideUp`
- Cấu hình file `src/main/resources/application.yml` với các biến môi trường:
 - `vnpay:`
 - `tmn-code: ${VNPAY_TMN_CODE:HUSN6OFW}`
 - `hash-secret: ${VNPAY_HASH_SECRET:LJW9JMRPZ55WB9E7XOKK3GZ6TADT2DLQ}`

- pay-url:
\${VNPAY_PAY_URL:https://sandbox.vnpayment.vn/paymentv2/vpcpay.html}
- return-url:
\${VNPAY_RETURN_URL:http://172.20.10.6:8080/rideUp/payments/vnpay/return}
- order-type: \${VNPAY_ORDER_TYPE:other}
- locale: \${VNPAY_LOCALE:vn}
- expire-minutes: \${VNPAY_EXPIRE_MINUTES:15}
- Chạy ứng dụng: mvn spring-boot:run
- Backend khởi động tại http://localhost:8080, WebSocket sẵn tại ws://localhost:8080/ws

4.2.2. Mobile App (React Native / Expo)

- Yêu cầu: Node.js 18+, npm/yarn, Expo CLI (npm install -g expo-cli)
- Vào thư mục frontend: cd Ride_Up/RideUpUI
- Cài dependencies: npm install (bao gồm @stomp/stompjs, sockjs-client cho WebSocket)
- Cấu hình BASE_URL trong src/config/config.js trỏ đến backend
- Chạy development server: npx expo start
- Quét QR bằng ứng dụng Expo Go hoặc nhấn A để mở Android Emulator

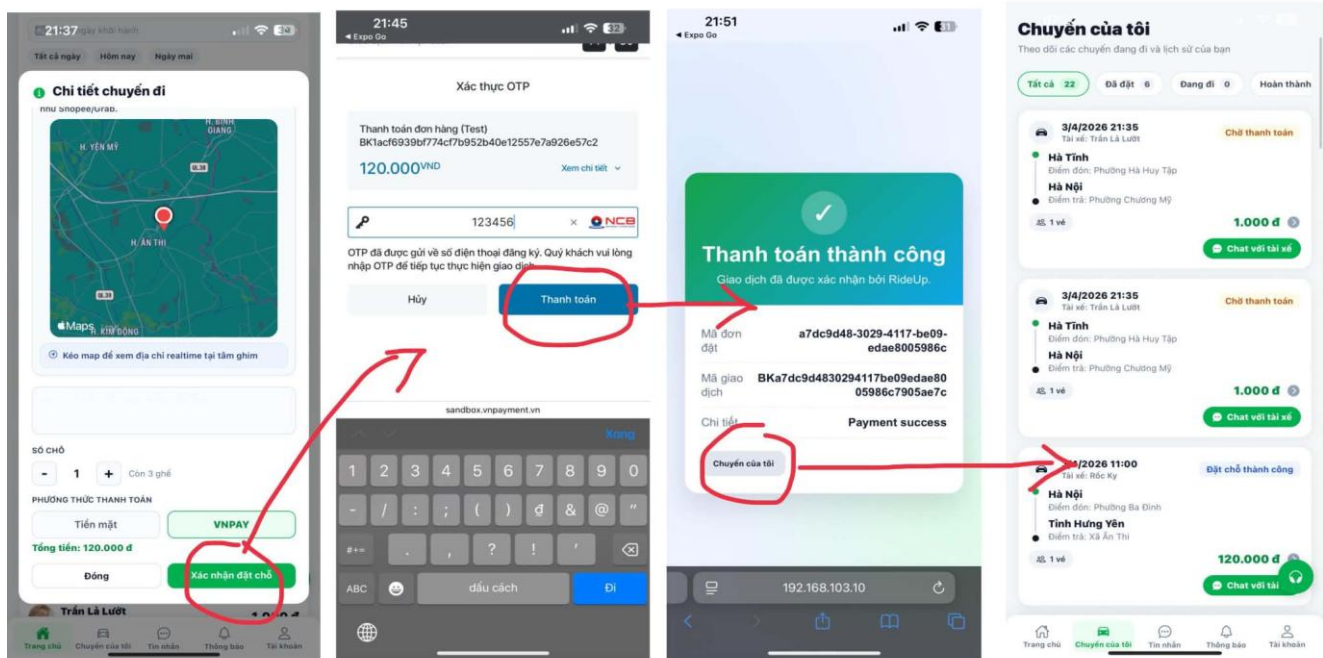
4.3. Kết quả thực hiện – Các chức năng Module cá nhân

4.3.1. Chức năng Thanh toán VNPay

Module cho phép khách hàng thanh toán chi phí chuyển đi một cách an toàn và tự động thông qua cổng thanh toán VNPAY.

- **Tính năng chính:** Sinh URL thanh toán với mã hóa bảo mật (HMAC-SHA512), tiếp nhận kết quả giao dịch tự động qua IPN Callback.
- **Điểm nổi bật:** Tự động đồng bộ trạng thái thanh toán với đơn đặt xe, hỗ trợ tính năng tự động hoàn tiền (Refund) khi chuyển đi bị hủy hợp lệ và

cơ chế tự động hủy đơn nếu khách hàng không thanh toán trong thời gian quy định (10 phút).



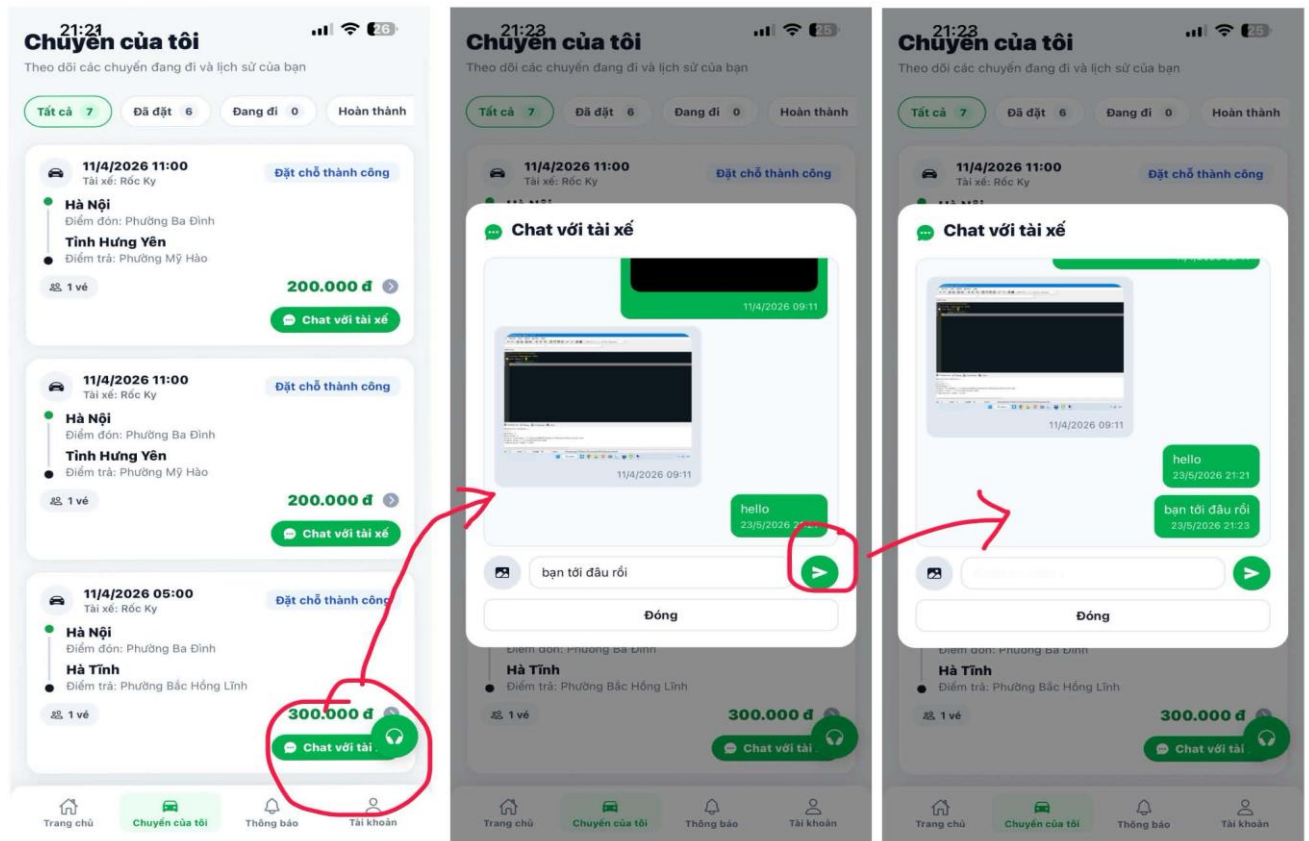
Hình 5. Luồng màn hình thanh toán VNPay trên ứng dụng

4.3.2. Chức năng Chat Realtime

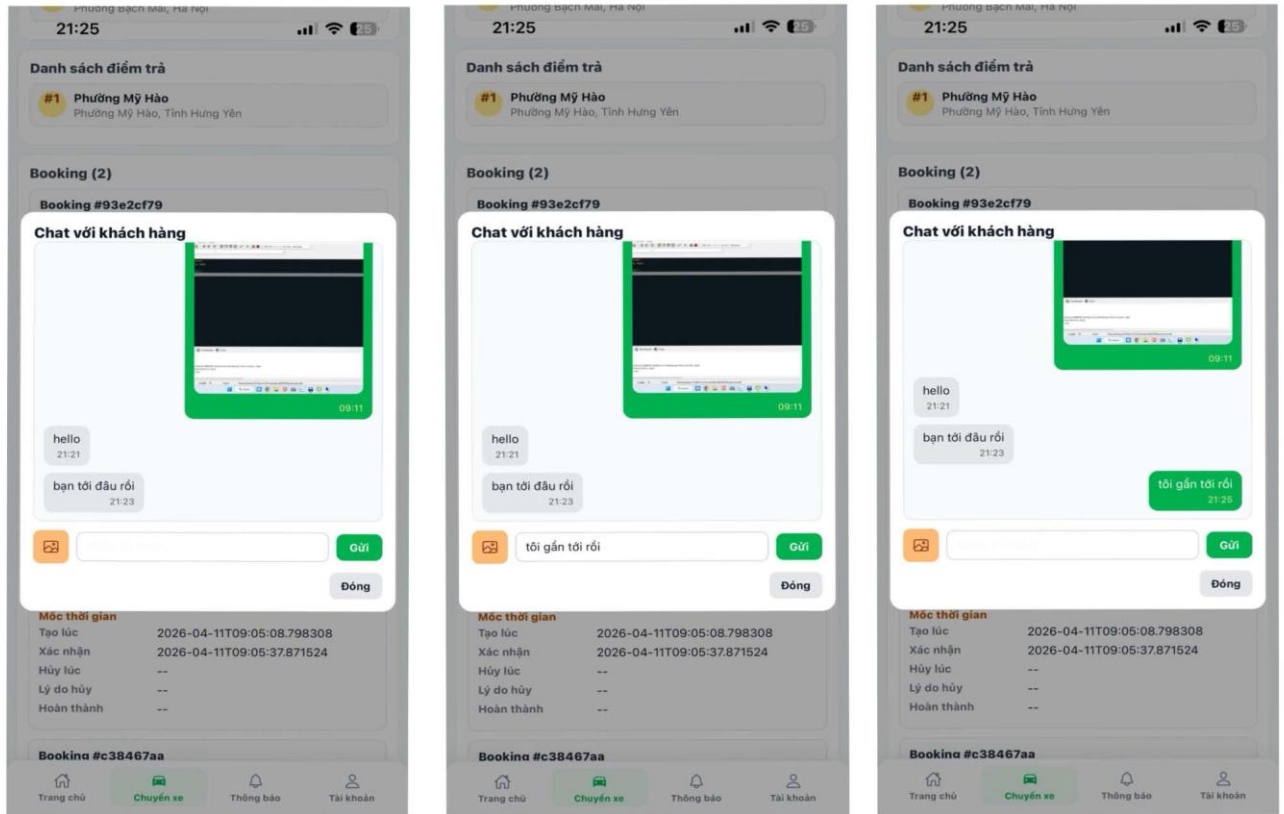
Xây dựng kênh liên lạc trực tiếp, tức thì giữa hành khách và tài xế thuộc cùng một chuyến xe, giúp hai bên dễ dàng trao đổi điểm đón, nhận dạng xe hoặc các lưu ý khác.

- **Công nghệ sử dụng:** Kết hợp WebSocket/STOMP (để truyền tải tin nhắn realtime) và cơ sở dữ liệu MongoDB (để lưu trữ và truy xuất lịch sử hội thoại với tốc độ cao).
- **Tính năng chính:** Hỗ trợ gửi tin nhắn văn bản và hình ảnh đính kèm. Kênh chat được thiết lập ràng buộc vòng đời chặt chẽ: chỉ mở khi đơn đặt

xe đã được xác nhận (Confirmed) và tự động đóng lại (khóa chat) khi chuyến đi hoàn thành hoặc bị hủy.



Hình 6. Luồng màn hình chat realtime phía khách hàng



Hình 7. Luồng màn hình chat realtime phía tài xế

4.3.3. Chức năng Thông báo Realtime

Hệ thống gửi thông báo đẩy (push notification) tức thời đến ứng dụng của người dùng (tài xế và khách hàng) mà không cần phải tải lại trang hay gọi API liên tục (polling).

- **Công nghệ sử dụng:** Spring Boot WebSocket kết hợp giao thức STOMP.
- **Tính năng chính:** Cập nhật ngay lập tức các sự kiện quan trọng của hệ thống như: có khách đặt chuyên mới, thanh toán thành công, tài xế xác

nhận nhận tiền mặt, hoặc chuyến đi bị hủy. Mỗi người dùng sẽ nhận thông báo qua một kênh (topic) định tuyến riêng biệt.



Hình 8. Màn hình thông báo realtime phía tài xế



Hình 9. Màn hình thông báo realtime phía khách hàng

4.4. Kết quả thử nghiệm

Chức năng	Kịch bản kiểm thử	Kết quả	Ghi chú
VNPay: sinh URL	Tạo booking VNPay với amount hợp lệ	Đạt	URL redirect đến VNPay sandbox thành công
VNPay: xác thực HMAC	Giả mạo chữ ký trong IPN callback	Đạt	Backend từ chối, không cập nhật payment
VNPay: IPN callback	VNPay sandbox gọi IPN sau thanh toán	Đạt	Payment.status = PAID, thông báo gửi đến khách
VNPay: refund	Tài xế hủy chuyến có booking VNPay	Đạt	Refund thành công, Payment.status = REFUNDED
Chat: mở thread	Mở chat giữa driver và customer trên booking	Đạt	Thread tạo với unique bookingId

Chat: gửi/nhận realtime	Gửi tin nhắn, kiểm tra client kia nhận ngay	Đạt	Tốc độ < 200ms
Chat: lịch sử	Load 100+ tin nhắn cũ bằng phân trang	Đạt	Query MongoDB theo compound index, nhanh
Chat: unread count	Gửi 5 tin, mở thread, kiểm tra badge reset	Đạt	Count = 0 sau khi đọc
Chat: đóng thread	Hoàn thành chuyển, thử gửi tin nhắn tiếp	Đạt	API trả lỗi, UI hiện "đã kết thúc"
Notification: push booking confirmed	Xác nhận booking, kiểm tra customer nhận	Đạt	Thông báo tức thì, đúng type và referenceId

Bảng 14. Kết quả kiểm thử

4.5. Dữ liệu thử nghiệm trong hệ thống

Loại dữ liệu	Số lượng
Số tài khoản đã đăng ký	12
Số hồ sơ tài xế đã duyệt (APPROVED)	5
Số chuyến đi đã tạo	36
Số chuyến đã hoàn thành	33
Số booking đã tạo	54
Số payment VNPay đã xử lý	28
Số refund VNPay thành công	6
Số chat thread đã tạo	35
Số tin nhắn chat trong hệ thống	247
Số tỉnh/thành phố trong hệ thống	34
Số xã/phường trong hệ thống	3321

Bảng 15. Dữ liệu thử nghiệm trong hệ thống

CHƯƠNG 5. KẾT LUẬN

5.1. Kết quả đạt được

Trong phạm vi cá nhân phụ trách module Thanh toán VNPay, Chat Realtime và Thông báo Realtime, bài tập lớn đã hoàn thành được các kết quả sau:

- Tích hợp đầy đủ cổng thanh toán VNPay: sinh URL thanh toán, xử lý IPN callback với xác thực HMAC-SHA512, hoàn tiền tự động khi hủy chuyển, query trạng thái giao dịch
- Xây dựng tính năng Chat Realtime hai chiều bằng WebSocket (STOMP protocol): lưu trữ tin nhắn trên MongoDB, đánh index tối ưu query, đóng thread theo vòng đời chuyển đi
- Triển khai hệ thống Thông báo Realtime theo topic cá nhân: không cần polling, push ngay khi có sự kiện (xác nhận đặt chuyển, chuyển hủy, thanh toán thành công, xác nhận tiền mặt)
- Thiết kế service layer rõ ràng (VnPayService, ChatService, ChatRealtimePublisher, NotificationRealtimePublisher) với dependency injection, dễ test và mở rộng
- Xử lý lỗi graceful: VNPay refund timeout không làm rollback việc hủy chuyển
- Giao diện React Native đồng nhất với toàn app: DriverMessagesScreen, ChatScreen, NotificationCenterScreen với WebSocket connect/disconnect lifecycle quản lý đúng

5.2. Hạn chế và điểm cần cải thiện

- Chat hiện chỉ hỗ trợ text và hình ảnh; chưa có voice message hay file đính kèm
- Thông báo realtime chỉ hoạt động khi app đang mở (foreground); chưa tích hợp Push Notification (FCM) để nhận khi app chạy nền
- Chưa có cơ chế reconnect tự động WebSocket khi mất kết nối mạng (cần implement retry logic)
- Lịch sử thông báo hiện chỉ lưu trong memory phía frontend; chưa persist vào database để hiển thị khi mở lại app
- VNPay IPN chưa có cơ chế retry cho trường hợp server down tạm thời khi VNPay gọi callback

5.3. Hướng phát triển

- Tích hợp Firebase Cloud Messaging (FCM) để gửi Push Notification khi app chạy nền hoặc tắt
- Bổ sung bảng notifications trong để lưu lịch sử thông báo, hỗ trợ đánh dấu đã đọc và load lịch sử

- Implement WebSocket reconnect tự động với exponential backoff cho môi trường mạng không ổn định
- Tích hợp thêm cổng thanh toán MoMo, ZaloPay để tăng lựa chọn cho hành khách
- Thêm tính năng chat nhóm (group chat) cho chuyến đi có nhiều hành khách
- Bổ sung read receipt (tin nhắn đã xem) trong chat thông qua WebSocket event

CHƯƠNG 6. PHỤ LỤC: BÁO CÁO KỸ THUẬT

Chương này trình bày chi tiết các lớp, hàm, bảng CSDL và API gọi ngoài phục vụ module Thanh toán VNPay, Chat Realtime và Thông báo Realtime. Toàn bộ code nằm trong project Spring Boot (backend) và React Native (mobile app).

6.1. Các lớp Entity và Bảng trong CSDL

6.1.1. Entity Payment – Bảng payment (MySQL)

File: RideUp/src/main/java/com/example/demo/entity/Payment.java

Mô tả: Lưu thông tin thanh toán cho từng booking (quan hệ 1-1 với Booking). Khi hoàn tiền, các trường refunded_at và provider_transaction_id được cập nhật. Bảng có index trên transaction_id để tra cứu nhanh callback từ VNPay.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	VARCHAR (UUID)	PK	Khóa chính, tự sinh UUID
booking_id	VARCHAR FK	FK→booking, UNIQUE	Liên kết 1-1 với booking
amount	DECIMAL(10, 2)	NOT NULL	Số tiền thanh toán (VND)
method	ENUM	NOT NULL	CASH BANK_TRANSFER VNPAY
status	ENUM	NOT NULL	UNPAID PAID REFUNDED FAILED
transaction_id	VARCHAR	INDEX	Mã GD nội bộ (vnp_TxnRef) dùng để tra cứu callback
provider_transaction_id	VARCHAR		Mã GD thực tế do VNPay trả về (vnp_TransactionNo)
paid_at	DATETIME		Thời điểm thanh toán thành công

refunded_at	DATETIME		Thời điểm hoàn tiền thành công
created_at	TIMESTAMP	NOT NULL	Thời điểm tạo bản ghi

6.1.2. Entity Booking – Bảng booking (MySQL)

File: RideUp/src/main/java/com/example/demo/entity/Booking.java

Trường	Kiểu dữ liệu	Mô tả
id	VARCHAR (UUID)	Khóa chính
customer_id	VARCHAR FK	Khách hàng đặt chỗ – dùng để xác thực quyền thanh toán
status	ENUM	PENDING → CONFIRMED khi thanh toán thành công
total_price	DECIMAL(10,2)	Tổng tiền = seatCount × pricePerSeat – dùng làm amount VNPay
payment	Payment (1-1)	Liên kết ngược tới Payment entity (cascade ALL)
confirmed_at	DATETIME	Set khi booking chuyển sang CONFIRMED sau thanh toán

6.1.3. ChatThreadDocument – MongoDB collection: chat_threads

File:

RideUp/src/main/java/com/example/demo/entity/chat/ChatThreadDocument.java

Trường	Kiểu dữ liệu	Mô tả
id	String (MongoDB _id)	Khóa chính MongoDB tự sinh
bookingId	String (Unique Index)	Liên kết 1-1 với Booking MySQL (unique)
tripId	String (Index)	Chuyến xe liên quan, dùng khi đóng thread theo trip
customerUserId	String	ID User MySQL của khách hàng

driverUserId	String	ID User MySQL của tài xế
status	ChatThreadStatus	ACTIVE – đang hoạt động; CLOSED – đã đóng (trip kết thúc/hủy)
chatAllowedFrom	LocalDateTime	Thời điểm thread được tạo/mở
chatAllowedUntil	LocalDateTime	Thời điểm đóng thread (null nếu ACTIVE)
lastMessageAt	LocalDateTime	Thời điểm tin nhắn gần nhất – dùng sắp xếp danh sách
lastMessagePreview	String	Hiển thị tin nhắn gần nhất (tối đa 120 ký tự)
customerUnreadCount	Integer	Số tin chưa đọc của khách hàng
driverUnreadCount	Integer	Số tin chưa đọc của tài xế

6.1.4. ChatMessageDocument – MongoDB collection: chat_messages

File:

RideUp/src/main/java/com/example/demo/entity/chat/ChatMessageDocument.java
va

Mô tả: Lưu từng tin nhắn trong luồng chat. Hỗ trợ 2 loại: MESSAGE (văn bản) và MEDIA (hình ảnh). Tin nhắn ảnh lưu URL của file trên Supabase Storage.

Trường	Kiểu dữ liệu	Mô tả
id	String	Khóa chính MongoDB
threadId	String (Index)	Luồng chat chứa tin nhắn
bookingId	String	Booking liên quan (dự thừa để truy vấn nhanh)
senderUserId	String	ID User MySQL của người gửi
senderRole	Role (ENUM)	CUSTOMER hoặc DRIVER
type	ChatMessageType (ENUM)	MESSAGE (văn bản) MEDIA (ảnh)
content	String	Nội dung văn bản (null nếu MEDIA)

imageUrl	String	URL ảnh Supabase (null nếu MESSAGE)
sentAt	LocalDateTime	Thời điểm gửi tin nhắn

6.2. Các Controller – Lớp xử lý HTTP Request

6.2.1. CustomerBookingController (phần Thanh toán VNPay)

File:

RideUp/src/main/java/com/example/demo/controller/CustomerBookingController.java

Method	Endpoint	Hàm xử lý	Chức năng
POST	/customer/bookings/{bookingId}/payments/vnpay-url	createVnpayPaymentUrl (bookingId, ip)	Tạo URL thanh toán VNPay cho booking. Yêu cầu @PreAuthorize isAuthenticated()
POST	/customer/bookings/{bookingId}/payments/confirm	confirmBookingPayment (bookingId, request)	Xác nhận thanh toán tiền mặt/chuyển khoản. Cập nhật Payment PAID và Booking CONFIRMED
GET	/payments/vnpay/return	vnpayReturnCallback (params)	Return URL: nhận redirect từ VNPay sau khi khách hoàn tất. Xác minh chữ ký, cập nhật trạng thái, trả HTML kết quả
GET	/payments/vnpay/ipn	vnpayIpnCallback (params)	IPN URL: server-to-server từ VNPay. Xác minh và cập nhật trạng thái idempotent
POST	/customer/bookings/{bookingId}/cancel	cancelBooking (bookingId, request, ip)	Hủy booking, tự động refund

			VNPay nếu đã thanh toán online
--	--	--	--------------------------------

6.2.2. ChatController

File: RideUp/src/main/java/com/example/demo/controller/ChatController.java

Base path: /chat. Annotation: @RestController @PreAuthorize()

Method	Endpoint	Hàm xử lý	Chức năng
POST	/chat/threads/open	openThread(request)	Mở/tạo thread chat từ bookingId. Thread tạo mới nếu chưa có, hoặc reopen nếu đã CLOSED
GET	/chat/threads	getMyThreads()	Lấy danh sách tất cả thread của user hiện tại (cả ACTIVE và CLOSED), sắp xếp theo lastMessageAt
GET	/chat/threads/{threadId}/messages	getMessages(threadId, limit)	Lấy lịch sử tin nhắn trong thread, mặc định 50 tin, tối đa 100
POST	/chat/threads/{threadId}/messages	sendMessage(threadId, request)	Gửi tin nhắn văn bản hoặc ảnh. Lưu MongoDB và publish STOMP realtime
POST	/chat/threads/{threadId}/read	markRead(threadId)	Đánh dấu đã đọc: cập nhật unread count về 0 cho user hiện tại

6.3. Các Service – Lớp xử lý nghiệp vụ

6.3.1. VnPayService

File: RideUp/src/main/java/com/example/demo/service/VnPayService.java

Tên hàm	Trả về	Mô tả chi tiết
buildPaymentUrl (txnRef, amount, ip, orderInfo)	String	1. Tạo URL thanh toán VNPay: 2. Build map params gồm vnp_Version, vnp_TmnCode, vnp_Amount, vnp_TxnRef, vnp_CreateDate, vnp_ExpireDate; Sắp xếp params theo alphabet 3. Ký HMAC-SHA512 4. Ghép URL + queryString + vnp_SecureHash
verifyReturn (requestParams)	boolean	Xác minh callback từ VNPay: 1. Lọc bỏ vnp_SecureHash 2. Sắp xếp các param còn lại 3. Tính HMAC-SHA512 4. So sánh case-insensitive với hash đã nhận
refund(txnRef, providerTxnId, amount, paidAt, ip, orderInfo)	RefundResult	Gọi VNPay Refund API: build JSON body có vnp_Command=refund, vnp_TransactionType=02, ký HMAC-SHA512; POST đến refundUrl; xử lý response lấy vnp_ResponseCode
queryTransaction (txnRef, transDate, ip, orderInfo)	QueryResult	Truy vấn trạng thái giao dịch từ VNPay trả về providerTransactionId và payDate để đồng bộ
generateTxnRef (bookingId)	String	Sinh mã GD: "BK" + bookingId + random 8 ký tự, tối đa 96 ký tự theo giới hạn VNPay
isConfigured()	boolean	Kiểm tra đủ cấu hình (tmnCode, hashSecret, payUrl, returnUrl) trước khi tạo URL thanh toán
hmacSha512 (key, data)	String	Ký dữ liệu bằng HMAC-SHA512, trả hex lowercase 128 ký tự

6.3.2. CustomerBookingService (phần thanh toán)

File: RideUp/src/main/java/com/example/demo/service/CustomerBookingService.java

Tên hàm	Mô tả chi tiết
createVnpayPaymentUrl(bookingId, ip)	Xác thực quyền sở hữu booking → gọi createVnPayPaymentUrlForBooking() tạo hoặc lấy Payment entity đang PENDING → gọi vnPayService.buildPaymentUrl() → trả Map gồm bookingId, paymentUrl, transactionRef
handleVnpayGatewayCallback(params, isIpn)	1. Tìm Booking theo vnp_TxnRef 2. Gọi vnPayService.verifyReturn() xác minh chữ ký 3. Kiểm tra idempotency (đã PAID rồi thì trả RspCode=02) 4. Gọi completeVnpayPayment() cập nhật Payment & Booking 5. Trả JSON cho IPN hoặc redirect data cho return URL
completeVnpayPayment(booking, success, transNo, payDate) [private]	Nếu success: Payment.status=PAID, paidAt=now(), providerTransactionId=vnp_TransactionNo; Booking.status=CONFIRMED, confirmedAt=now(); tạo chat thread; gửi STOMP notification PAYMENT_SUCCESS. Nếu fail: Payment.status=FAILED; gửi thông báo PAYMENT_FAILED
confirmBookingPayment(bookingId, request)	Xác nhận thanh toán tiền mặt/chuyển khoản thủ công: validate method=BANK_TRANSFER và status=UNPAID → Payment.status=PAID → Booking.status=CONFIRMED → chatService.ensureThreadForConfirmedBooking()
tryAutoRefundVnPay(booking, ip, reason) [private]	Kiểm tra payment.method=VNPAY và status=PAID → gọi vnPayService.refund() → nếu thành công: payment.status=REFUNDED, refundedAt=now(). Lỗi refund chỉ log warning, không dừng luồng hủy
cancelMyBooking(bookingId, request, ip)	Validate status PENDING/CONFIRMED và không quá 1 giờ trước khởi hành → Booking.status=CANCELLED_BY_CUS

	TOMER → restoreTripSeat() → tryAutoRefundVnPay() → closeThreadByBookingId() → gửi notification BOOKING_CANCELLED
--	---

6.3.3. ChatService

File: RideUp/src/main/java/com/example/demo/service/ChatService.java

Service trung tâm xử lý toàn bộ nghiệp vụ chat. Tất cả phương thức đều @Transactional. Kết hợp MySQL (để validate Booking) và MongoDB (để lưu ChatThread, ChatMessage).

Tên hàm	Mô tả chi tiết
openThreadByBooking(bookingId)	Validate user là thành viên booking → isChatAllowed() (booking/trip chưa hoàn thành/hủy) → tìm hoặc tạo thread → nếu CLOSED thì reopen → trả ChatThreadResponse với tiêu đề từ tuyến đường và tên đối phương
ensureThreadForConfirmedBooking(bookingId)	Gọi sau khi booking được xác nhận thanh toán: tự động tạo hoặc mở thread, dùng trong confirmBookingPayment()
sendMessage(threadId, request)	1. Validate thread active và user là thành viên 2. isChatAllowed() – đóng thread nếu trip đã kết thúc; 3. Tạo ChatMessageDocument với type MESSAGE/MEDIA 4. Lưu MongoDB 5. Cập nhật thread.lastMessageAt, lastMessagePreview, unreadCount 6. Publish STOMP via chatRealtimePublisher.publishThreadMessage()
getMyThreads()	Load threads theo role (CUSTOMER/DRIVER); de-duplicate; batch load Bookings; resolve effective

	status; sắp xếp theo lastMessageAt giảm dần. Tối ưu tránh N+1 query
getMessages(threadId, limit)	Lấy tối đa 100 tin nhắn gần nhất (mặc định 50) reverse để hiển thị cũ → mới
markThreadRead(threadId)	Đánh dấu số tin nhắn đã đọc bằng 0 khi đã đọc tin nhắn
closeThreadsByTripId(tripId, reason)	Đóng toàn bộ ACTIVE thread của một trip; gọi khi trip hoàn thành hoặc hủy
closeThreadByBookingId(bookingId, reason)	Đóng thread của một booking cụ thể; gọi khi hủy booking
isChatAllowed(booking)	Kiểm tra booking.status có thuộc {CANCELLED, COMPLETED} và trip.status có thuộc {COMPLETED, CANCELLED}

6.3.4. ChatRealtimePublisher

File:

RideUp/src/main/java/com/example/demo/service/ChatRealtimePublisher.java

Tên hàm	Mô tả chi tiết
publishThreadMessage(ChatMessageResponse message)	Gọi SimpMessagingTemplate.convertAndSend("/topic/chat.thread.{threadId}", message). Chỉ publish khi threadId không rỗng. Tất cả subscriber của topic này (customer và driver) nhận được tin nhắn ngay lập tức

6.3.5. NotificationRealtimePublisher

File:

RideUp/src/main/java/com/example/demo/service/NotificationRealtimePublisher.java.

Tên hàm	Mô tả chi tiết
notifyUser(targetUserId, type, title, message, referenceId)	Build RealtimeNotificationResponse với id mới (UUID), type, title, message, referenceId, createdAt → publish đến /topic/notifications.user.{targetUserId}

	qua SimpMessagingTemplate. Bỏ qua nếu targetUserId, title hoặc message rỗng
--	---

Các loại type thông báo trong module cá nhân:

Type	Gửi đến	Trigger
PAYMENT_SUCCESS	Customer	Thanh toán VNPay thành công (callback)
PAYMENT_FAILED	Customer	Thanh toán VNPay thất bại (callback)
BOOKING_CONFIRMED	Customer	Booking chuyển sang CONFIRMED sau thanh toán
BOOKING_CANCELLED_BY_CUSTOMER	Customer	Khách hủy booking thành công
CASH_PAYMENT_CONFIRMED	Customer	Tài xế xác nhận đã thu tiền mặt
BOOKING_NEW	Driver	Có khách đặt chỗ mới trên chuyến

6.4. Các Repository – Lớp truy cập CSDL

6.4.1. ChatThreadRepository (MongoDB)

File:

RideUp/src/main/java/com/example/demo/repository/ChatThreadRepository.java

a

Tên hàm	Mô tả
findByBookingId(bookingId)	Tìm thread theo bookingId (unique index), dùng khi mở thread hoặc đóng theo booking

<code>findByTripIdAndStatus(tripId, status)</code>	Tìm tất cả thread ACTIVE của một trip, dùng khi hoàn thành/hủy chuyến
<code>findByCustomerUserIdAndStatusInOrderByLastMessageAtDesc(userId, statuses)</code>	Lấy thread của khách hàng theo trạng thái, sắp theo tin nhắn mới nhất
<code>findByDriverUserIdAndStatusInOrderByLastMessageAtDesc(userId, statuses)</code>	Lấy thread của tài xế theo trạng thái, sắp theo tin nhắn mới nhất

6.4.2. ChatMessageRepository (MongoDB)

File:

RideUp/src/main/java/com/example/demo/repository/ChatMessageRepository.java

va

Tên hàm	Mô tả
<code>findByThreadIdOrderBySentAtDesc(threadId, pageable)</code>	Lấy tin nhắn của thread theo trang, sắp xếp mới nhất trước; kết quả được reverse để hiển thị cũ → mới

6.4.3. BookingRepository (MySQL – liên quan thanh toán)

File:

RideUp/src/main/java/com/example/demo/repository/BookingRepository.java

Tên hàm	Mô tả
<code>findByIdWithPayment(bookingId)</code>	@Query JPQL với JOIN FETCH payment – load eager Payment để tránh LazyInitializationException khi xử lý thanh toán
<code>findByPaymentTransactionId(txnRef)</code>	Tìm Booking theo vnp_TxnRef từ callback VNPay, dùng trong <code>handleVnpayGatewayCallback()</code>

6.5. Các DTO (Data Transfer Object)

Tên DTO	Loại	Mô tả các trường chính
---------	------	------------------------

CreateBooking Request	Request	tripId, pickupPointId, dropoffPointId, seatCount, paymentMethod , passengerName, contactPhone, pickupAddress, pickupLat/Lng
ConfirmPayment Request	Request	transactionId (optional – mã GD thủ công cho chuyển khoản)
OpenChatThread Request	Request	bookingId – ID booking muốn mở thread chat
SendChatMessage Request	Request	content (văn bản, tối đa 2000 ký tự), imageUrl (URL ảnh Supabase, tối đa 3000 ký tự), type (MESSAGE/MEDIA – optional, tự suy luận nếu không có)
CustomerBooking Response	Response	id, status, seatCount, price, from, to, pickupPoint, dropPoint, departureTime, driverName, driverRating, paymentMethod, paymentStatus, paymentUrl, paymentTransactionRef, hasRated
ChatThread Response	Response	id, bookingId, tripId, chatTitle, status, lastMessageAt, lastMessagePreview, myUnreadCount, customerUnreadCount, driverUnreadCount
ChatMessage Response	Response	id, threadId, bookingId, senderUserId, senderRole, type, content, imageUrl, sentAt
RealtimeNotification Response	Response / WebSocket	id, type, title, message, targetUserId, referenceId, createdAt

6.6. Các API Gọi Ngoài (External API)

6.6.1. VNPay Payment Gateway

Thông số	Giá trị
Pay URL (sandbox)	https://sandbox.vnpayment.vn/paymentv2/vpcpay.html

6.6.2. WebSocket STOMP – Chat và Thông báo Realtime

Đường dẫn	Module	Khi nào publish
/topic/chat.thread.{threadId}	Chat	Khi có tin nhắn mới trong thread; cả customer và driver đều nhận
/topic/notifications.user.{userId}	Notification	Mọi sự kiện liên quan đến user: thanh toán, booking, chuyển đi